

Projet MOGPL : La balade du robot

Lionel Alves Vieira and Thomas Lu

Sorbonne Université

Introduction

Ce projet consiste à modéliser et résoudre le problème d'un déplacement d'un robot sur une grille comportant des obstacles. Le robot dispose d'une **orientation** et peut exécuter **trois** types d'actions : *avancer*, *tourner à gauche* ou *tourner à droite*. L'objectif est de déterminer la **séquence d'actions minimisant la distance entre un point de départ et un point d'arrivée**.

Nous répondrons successivement aux questions (a) à (e) du sujet.

1 Modélisation du problème comme graphe orienté

On modélise le déplacement du robot comme un problème de plus court chemin dans un graphe orienté $G = (V, E)$.

Ensemble des sommets

Un état est représenté par un triplet :

$$(r, c, k)$$

où :

$$(r, c) \in \llbracket 0, M \rrbracket \times \llbracket 0, N \rrbracket, \quad k \in \{0, 90, 180, 270\}$$

k représentant l'orientation du robot :

$$\begin{array}{ll} 0 & \rightarrow \text{Est,} \\ 90 & \rightarrow \text{Nord,} \\ 180 & \rightarrow \text{Ouest,} \\ 270 & \rightarrow \text{Sud.} \end{array}$$

En pratique, dans le code, on utilisera une notation $\{0, 1, 2, 3\}$ pour remplacer $\{0, 90, 180, 270\}$. On note V l'ensemble de tous les triplets (r, c, k) tels

que le robot puisse physiquement occuper l'intersection (r, c) , c'est à dire, aucune des quatre cases du bloc 2×2 associé n'est un obstacle.

$$V = \{(r, c, k) \mid (r, c) \text{ valide et } k \in \{0, 90, 180, 270\}\}.$$

Transitions (arcs)

Depuis un sommet $u = (r, c, k)$, on peut appliquer trois types d'actions :

- *tourner à droite* : $k \mapsto (k - 90) \bmod 360$;
- *tourner à gauche* : $k \mapsto (k + 90) \bmod 360$;
- *avancer de l cases* ($l \in \{1, 2, 3\}$) suivant l'orientation.

Les positions atteintes en avançant dépendent du cas :

Si $k = 0$ (Est)

$$v_1 = (r, c+l, k), \quad v_{k+1} = (r, c+(k+90) \bmod 360, k), \quad v_{k+2} = (r, c+(k+180) \bmod 360, k).$$

Si $k = 90$ (Nord)

$$v_1 = (r-l, c, k), \quad v_{k+1} = (r, c+(k+90) \bmod 360, k), \quad v_{k+2} = (r, c+(k+180) \bmod 360, k).$$

Si $k = 180$ (Ouest)

$$v_1 = (r, c-l, k), \quad v_{k+1} = (r, c+(k+90) \bmod 360, k), \quad v_{k+2} = (r, c+(k+180) \bmod 360, k).$$

Si $k = 270$ (Sud)

$$v_1 = (r+l, c, k), \quad v_{k+1} = (r, c+(k+90) \bmod 360, k), \quad v_{k+2} = (r, c+(k+180) \bmod 360, k).$$

Chaque mouvement n'est autorisé que si toutes les positions intermédiaires sont valides.

Ensemble des arcs et coût

On définit :

$$E = \{(u, v) \in V^2 \mid u \text{ et } v \text{ sont liés par une action valide}\}.$$

Chaque action coûte exactement 1 :

$$c(u, v) = 1.$$

Objectif

Résoudre le problème revient à trouver le plus court chemin entre l'état initial :

$$v_0 = (r_D, c_D, k_D)$$

et un état final :

$$(r^*, c^*, k), \quad k \in \{0, 90, 180, 270\}.$$

2 Algorithme de résolution & complexité

Comme tous les arcs ont le même coût unitaire (1), un **BFS** (Breadth First Search) permet de trouver un plus court chemin optimal.

Un état de la BFS est :

$$(r, c, k)$$

correspondant exactement à un sommet du graphe. La BFS est particulièrement adaptée ici car :

- toutes les actions ont un **coût unitaire** ;
- la BFS explore les états par couches croissantes ;
- dès que l'on atteint l'état final, la solution trouvée est **garantie optimale**.

Contrairement à un parcours exhaustif du graphe théorique G , nous ne construisons jamais explicitement G : les voisins sont générés **dynamiquement** au moment de l'exploration. Cela permet de réduire grandement la mémoire utilisée.

Transitions explorées

Depuis chaque état, la BFS explore jusqu'à cinq transitions :

- tourner à gauche (G)
- tourner à droite (D)
- avancer de 1 case (a1)
- avancer de 2 cases (a2)
- avancer de 3 cases (a3)

Complexité

Le nombre maximal d'états est :

$$|V| \leq 4MN$$

avec M le nombre de colonnes et N le nombre de lignes. Chaque état génère au plus 5 successeurs. Ainsi, la complexité est :

$$O(|V|) = O(MN),$$

soit linéaire en le nombre d'états

3 Performances en fonction de la taille de la grille

Nous mesurons les temps d'exécution pour des grilles de tailles :

10, 20, 30, 40, 50

avec un nombre d'obstacles proportionnel à la taille (10 obstacles pour 10×10 , etc.).

Pour chaque taille, 10 instances aléatoires ont été générées et le temps moyen a été enregistré dans un fichier CSV.

Les valeurs exactes se trouvent dans :

— `stats_10-50.csv`

4 Performances en fonction du nombre d'obstacles

Sur une grille fixe de taille 20×20 , nous faisons varier le nombre d'obstacles :

10, 20, 30, 40, 50.

10 instances sont générées par valeur.

Analyse

Avec peu d'obstacles, la BFS explore un espace plus vaste $\rightarrow$ temps plus long.

Avec beaucoup d'obstacles, certaines zones deviennent inaccessibles $\rightarrow$ BFS termine plus rapidement.

Les valeurs exactes sont fournies dans :

— `stats_10-50_obstacles.csv`.

5 Génération aléatoire d'une grille via un programme linéaire

On souhaite tirer aléatoirement P obstacles sur une grille (M, N) en respectant plusieurs contraintes. On attribue d'abord un poids entier $x_{ij} \in [0, 1000]$ à chaque case. On cherche ensuite à choisir les P cases d'obstacles minimisant la somme des poids.

Variables

$p_{ij} \in \{0, 1\}$ vaut 1 si la case (i, j) est un obstacle.

Objectif

$$\min \sum_{i=1}^M \sum_{j=1}^N x_{ij} p_{ij}$$

Contraintes

1. Nombre exact d'obstacles :

$$\sum_{i,j} p_{ij} = P$$

2. Limitation par colonne :

$$\sum_{i=1}^M p_{ij} \leq \frac{2P}{N} \quad \forall j \in \llbracket 1, N \rrbracket.$$

3. Limitation par ligne :

$$\sum_{j=1}^N p_{ij} \leq \frac{2P}{M} \quad \forall i \in \llbracket 1, M \rrbracket.$$

4. Interdiction des séquences 101

Pour toute ligne :

$$p_{i,j} - p_{i,j-1} - p_{i,j+1} \geq -1 \quad \forall (i, j) \in \llbracket 2, M-1 \rrbracket * \llbracket 1, N \rrbracket.$$

Pour toute colonne :

$$p_{i,j} - p_{i-1,j} - p_{i+1,j} \geq -1 \quad \forall (i, j) \in \llbracket 1, M \rrbracket * \llbracket 2, N-1 \rrbracket.$$

5. Contraintes de signes :

$$p_{i,j} \in \{0, 1\} \quad \forall (i, j) \in \llbracket 1, N \rrbracket \times \llbracket 1, M \rrbracket.$$

Interface utilisateur

Une interface simple (via **Scanner**) permet à l'utilisateur de :

- choisir M, N, P ;
- générer la grille optimisée via Gurobi ;
- choisir départ, orientation et arrivée ;
- lancer le solver BFS ;
- afficher le chemin optimal.

Conclusion

Nous avons :

- modélisé le problème comme un graphe orienté ;
- implémenté un algorithme optimal en temps linéaire $O(MN)$;
- réalisé des expérimentations complètes ;
- formulé un programme linéaire permettant de générer des grilles réalistes selon des contraintes structurelles ;
- proposé une interface utilisateur complète.